A working day with Claude Code

From the first prompt to the next day

Florian Gärtner

Where we are today

	What this course is for
	Getting it running
	Try it out
	The terminal, and how to read what Claude does
	Lunch
→	**A working day with Claude Code** ← you are here
	What the model reads, predicts and weighs
	A model, a menu, and a loop
	It doesn’t remember. It’s handed everything again.
	Wrap, done at 17:00

Learning objectives

After this block you will be able to:

1. **Set up** a session before the first prompt: the right folder (or `/cd` there), a name, foreground or background, the model and effort, how much Claude may do alone, and a plan.
2. **Resume** yesterday's session, or find one that seems lost.
3. **Steer** a running session: point Claude at a file, run a command yourself, stop it, and see what Claude actually did.

What changed since lunch

Start Claude Code again: the kit loads when Claude Code starts. Nothing from this morning is lost. Your files stay where they are, and this morning's session can be resumed, kit included. Later in this block we show how.

Then you will see:

- **Two lines under the input box:** the folder (and the session's name, once it has one), then the model and how full the session is.
- **A line from Claude before a task**, naming the skills it loaded for it.

And one thing to type when you stop: `/session-end`. Claude writes down where you stopped and what is still open, then suggests `/clear`. If your project already uses git, it also commits, and asks before it pushes. New to git? Answer **No** to the push until block 2_5.

Green marks what your kit brings, on top of Claude Code. Take it with you when you change computers.

A working day

What a day with Claude Code looks like

A morning in one folder, in plain requests:

- *"Read the three papers in `literature/` and tell me how each one measures financial literacy."*
- *"Tighten the second paragraph of the introduction in `main.tex`. Keep my wording where you can."*
- *"`analysis.R` stops with an error. Fix it."*
- *"Run the analysis again and tell me what changed in Table 2."*

Claude works on papers, notes and drafts in any folder, as much as on scripts.

You ask. Claude writes. You read it and decide.

From the folder to the work

The folder
where the work lives
start Claude Code there

→

The session
pick up, or start new
one topic each

→

The settings
model, effort, mode
a plan before Claude acts

→

The work
show, steer, stop
see what Claude did

Three choices before the first request. Then you steer.

Before you start

Start Claude Code in the folder the work lives in

```
cd ~/research/my_paper
claude
```

TERMINAL

```
cd ~\research\my_paper
claude
```

TERMINAL

Claude Code works in the folder you start it in. Claude Code reads that folder's `CLAUDE.md` and settings, and files the session under that folder.

It is close to an R project: one folder, one piece of work.

> ⚠ **Started somewhere else**
>
> Claude Code then works without that folder's `CLAUDE.md` and settings. A command that acts on "the current folder" uses the one you started in.

Already running somewhere else?

```
/cd /Users/you/research/my_paper
```
CLAUDE CODE

```
/cd C:/Users/you/research/my_paper
```
CLAUDE CODE

`/cd` moves the running session to another folder. It keeps the conversation and loads that folder's `CLAUDE.md` and settings. Type part of the path, and `Tab` completes it.

> ⚠ **The first time, Claude Code asks whether you trust the folder**
>
> A folder's settings can run commands of their own. Say yes only for folders whose contents you know.

Pick up where you left off

	What it resumes
`claude --continue`	the most recent session **in this folder**
`claude --resume`	a list of **this folder's** sessions
`/resume`	the same list, from inside a session

`/rename` a session before you leave it, so you can find it in the list.

> ⚠ `--continue` **takes the most recent, whatever it was**
>
> If your last session in this folder was a quick throwaway, that is what comes back. After a mishap, use `--resume` and pick.

Your session is on disk

Every session is saved as a file, filed under its folder: the one you started in, or the one you moved it to with `/cd`.

A session that seems lost is usually filed under a different folder. In the list, `Ctrl+A` shows every folder's sessions and the folder each one belongs to. Start Claude Code in that folder, and the session is in the list.

Claude Code deletes these session files after 30 days by default. Save anything you want to keep longer in the folder the work lives in.

Let Claude suggest the name and the folder

A name shows on the prompt bar, so two sessions side by side stop looking alike, and tomorrow you find it by that name in the `/resume` list.

Only you can type `/rename` and `/cd`. The kit asks Claude to suggest both when they are due, as a line you paste. If no line comes, type them yourself.

`/rename` on its own also works. Claude Code then picks a name from the conversation.

One topic per session

A session carries everything it has seen into every later turn.

Finish one piece of work, then start fresh for the next. Otherwise the first task’s files and dead ends stay in front of the model while it works on the second.

A session with one topic is also easy to name and find again.

Several sessions at once

You can run more than one session, each in its own terminal window. All of them draw on the same plan limits, or on the same budget if you use an API key.

How we split them is our way of working, not a Claude Code feature:

	You	Claude
Foreground	watch, answer, decide	asks when unsure
Background	look in when it reports	works on without you

> **Rule of thumb**
>
> Decide in the foreground. Give a background session only work it can finish without asking you.

Decide first, then let it run

For a large project with many open questions:

1. A **decision session**: walk the open questions with Claude, one at a time. Claude writes each decision into a file in the project.
2. A **background session**: builds the decided items from that file, one after another.

A background session that hits a new question writes it down for the next decision session instead of guessing.

In one folder, everything uncommitted is shared. So let one session at a time write in a project.

Which model for what

Model	For	API price per million input tokens
Haiku	simple tasks, such as look-ups or summaries	from $0.10
Sonnet	everyday work	$2
Opus	complex reasoning	$4
Fable	the hardest and longest tasks	$10
Mythos	Fable, for approved security work only	$10

Until you choose another, Claude Code uses Opus.

Choose the model and the effort

	Command	Shortcut
Model	`/model`	`Alt+P` (macOS: `Option+P`)
Effort	`/effort`	

Effort decides how much Claude thinks before it answers. Save the larger models, and higher effort, for hard problems.

Your choice stays for later sessions, except effort `max`, which lasts one session. Press `s` in the picker to change it for this session only.

> **The largest lever you control**
>
> The model you pick decides cost and speed more than any other setting.

Permission modes

Mode	What Claude does without asking
Manual	reads
Accept edits	reads, edits files and runs simple file commands
Auto	everything, while a second model reviews the actions instead of you
Plan	reads and proposes a plan, but edits nothing (next slide)

`Shift+Tab` switches. Sessions start in Auto if the model supports it. Current models do. An older one, such as Haiku 4.5, starts in Manual.

A background session needs Auto. In the other modes it stops at its first question and waits for you. Your settings can still make Claude ask in Auto: with the kit, a delete command or a push does.

Let Claude plan before it acts

In **plan mode** Claude reads, explores and proposes a plan, and edits nothing until you approve. Use it when a task touches several files, or when you are not yet sure what you want.

To enter plan mode	
`Shift+Tab` until the status bar shows `⏸ plan mode on`	mid-session
`/plan` in front of one prompt	one prompt only
`claude --permission-mode plan`	from the start

When the plan is ready you choose: approve and let Claude work, approve and review each edit, or keep planning.

What is Claude for in this session?

Role	You ask for	Set up the session
Coder	a script, a pipeline step	Claude edits, plan mode first for a big task
Discussion partner	a design, open questions	one question at a time, decisions in a file
Reviewer	the flaws in your work	a fresh session that did not write it
Writer / editor	a draft, a presentation, tighter prose	discuss each change with Claude, check it in the PDF or on the slides
Research assistant	papers, data, answers from your PDFs	a quote and a page per claim, and numbers from a script
Measuring device	labels for your text data	check against a sample you coded by hand
Tutor	an explanation of the code, method or error	ask for the explanation before the fix
Secretary	what is due, an email draft	a deadlines file, Claude drafts, you send

Pick the role before the first prompt, since it decides how you set up the session.

Set up your own session

Your project · free time

- Is the session in your project's folder? If not, type `/cd` and the folder.
- Let Claude suggest a name, and type `/rename` with it.
- Optional: your profile, `about-me.md`, tells Claude who you are: field, language, tools. It goes to the AI provider with every request, so leave out what you would not send them. If Claude has not interviewed you for it yet:

> Interview me to fill in ~/.claude/about-me.md from the template, one question at a time.

While you work

Show Claude what you mean

You want Claude to look at your cleaning script, or at a figure that came out wrong. Describing either in words is slow, and details get lost.

Claude only knows what is in the session. Put the thing itself in:

You	Claude gets
type `@main.tex`	the whole file, at once
type `@literature/`	the list of files in the folder, not their contents
drag a file into the window	its path, as text. Claude opens the file when it needs to.
copy a screenshot, then press `Alt+V` on Windows, `Ctrl+V` on macOS and Linux	the image

Type `@` and a menu of paths opens. A dragged image arrives as the image itself. On Windows, `Ctrl+V` or a right-click pastes text.

Run a terminal command yourself

```
1  ! git status
```
CLAUDE CODE

A line that starts with `!` runs a **terminal** command, as if you had typed it in the terminal. Claude Code's own commands, such as `/model`, start with `/` instead.

Claude is not asked first. The command and its output go into the session, and Claude replies to them.

Most often, Claude asks you to: a command that waits for you, such as a login that wants your password, which Claude cannot type. Typed with `!`, its output lands in the session. Also useful when you know a command and want Claude to see the result, such as an error message.

Stopping and steering

You	What happens
press `Esc`	Claude stops what it is writing or running. Work already done stays.
send a message while Claude runs a command or edits a file	The message waits until that step is done. Claude then reads it and carries on, without stopping.

⚠ **Stopped is not undone**

A command you interrupt may already have run. Check before you assume nothing happened.

See what Claude actually did

On screen, each step shows Claude's own label for it, such as *"Run the cleaning script"*, and the start of the command.

`Ctrl+O` shows the exact command, its full result, and which model answered. `Ctrl+O` again, or `Esc`, closes it.

> **Rule of thumb**
>
> When Claude says it did something, `Ctrl+O` shows what it did.

Side questions and long commands

`/btw` asks a quick question about the session without adding it to the session. Claude sees everything so far but cannot use tools. Each `/btw` gets one answer. To follow up, ask another `/btw`: it sees your earlier side questions.

`Ctrl+B` moves a long-running command to the background: it keeps running on your computer, but Claude stops waiting for it. You and Claude can go on, and the command's output goes to a file Claude can read. `/tasks` lists what is running.

Send the reading away

Reading twenty files fills your session with twenty files.

> *"Use a subagent to find where the survey weights are computed."*

A **subagent** reads in its own separate session and sends back a summary. Your session gets the answer without the pages.

Claude often does this on its own. To search a folder, Claude sends out an **Explore** subagent. You see a row such as `Explore(Find the survey weights)`.

Rule of thumb

A big reading job goes to a subagent.

What to take away

1. Put the session where the work is, and give it a name. Only you can type `/cd` and `/rename`.
2. Show Claude what you mean: `@` a file, paste a screenshot, `!` a command.

Point Claude at your own files

Your project · free time

- In your next request, name a file with `@`.
- Run a command yourself with `!`, and let Claude reply to its output.
- When Claude says it did something, press `Ctrl+O` and read what it did.